Linux Notes: Links, Disk Usage and Finding Files

Hard links, soft links, inodes, du, df, find and locate

What is inside

1. Inodes, hard links and soft links, with exact commands
2. Checking inode numbers, and what happens when you delete links
3. du and df: how much space is used and how much is free
4. find by name, size and time, and find versus locate
5. Command chart and interview answers (with sources on the last page)

How to practise safely

The exact link-creation commands from class use /root/cook.txt and /opt: they are shown as a root demonstration, not for practice. Every example you run yourself works inside a brand new throw-away folder made by mktemp, with no sudo. The only deleting commands are rm -i and rmdir on the demo files, and each one is marked in red.

```
demo=$(mktemp -d) && cd "$demo" && pwd
echo "hello linux" > notes.txt
```

The exact commands from class

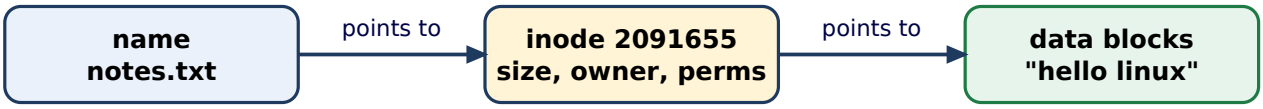
Soft link: `ln -s /root/cook.txt /opt`

Hard link: `ln /root/cook.txt /opt`

Both create `/opt/cook.txt`, so they are alternatives: run one, not both. They need root privileges, an existing /opt, and no cook.txt inside it yet. Sections 2 and 3 show the full demo, the checks and the caveats.

1. What is an inode?

Every file on a Linux file system has an **inode**. The inode stores the file details (owner, size, permissions, times) and where the data lives. It has a number. The **file name** in a folder is just a label that points to an inode.



A file name is only a label. The inode holds the file details, and points to the data.

ls -i shows the inode number. Two names with the same inode number are the same file.

This is the key idea behind links. A **hard link** is one more name for the same inode. A **soft link** is a separate small file that stores a path.

> **From the manual (symlink(7))**
> Each hard link is a reference to the same inode number. Changes to a file are independent of the name used to reference it.

2. Hard link

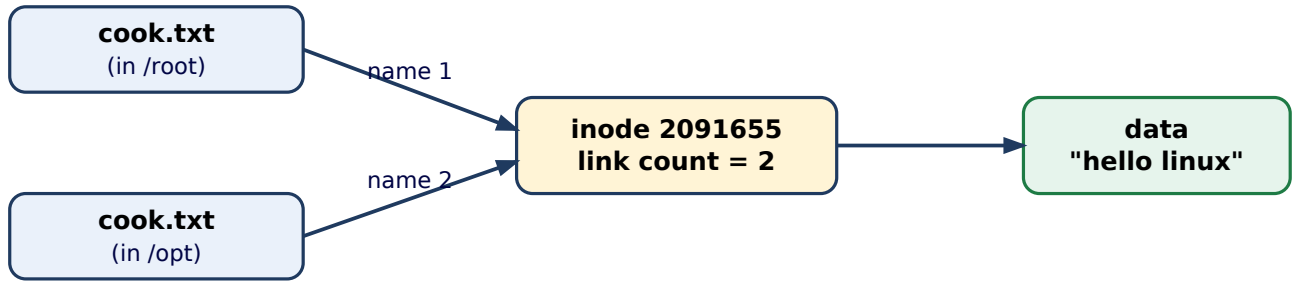
Syntax: **`ln TARGET LINK_NAME`** . With no option, **ln** makes a hard link. If LINK_NAME is an existing folder, the link goes inside it with the same file name, and the target must already exist (ln(1)).

```
ln /root/cook.txt /opt
ls -i /root/cook.txt /opt/cook.txt
```

```
2091655 /opt/cook.txt
2091655 /root/cook.txt
```

The new name `/opt/cook.txt` appeared inside `/opt`. **The inode numbers** match: both names are the same file. **ls -i** prints the inode number of each name (ls(1)). **ls -l** would also show the **link count** jump to 2.

Root demonstration
These paths need root privileges: a normal user cannot read /root or write into /opt. /opt must exist, and /opt/cook.txt must not exist yet. A hard link also needs /root and /opt on the same file system. Practise the same idea safely in your demo folder: **`ln notes.txt hard.txt`** and **`ls -i notes.txt hard.txt`**



Why use hard links

- A second name for the same file, without copying its data. No extra space used.
- The data stays alive while any name is left: deleting one name never loses the file.
- One file reachable from two folders; edit it through either name.

Limits of hard links

Limit	Why
Not for directories	Stops loops in the folder tree (symlink(7))
Not across file systems	Inode numbers are not unique across file systems (symlink(7))
Target must exist	ln needs an existing file to hard link (ln(1))

3. Soft link (symbolic link)

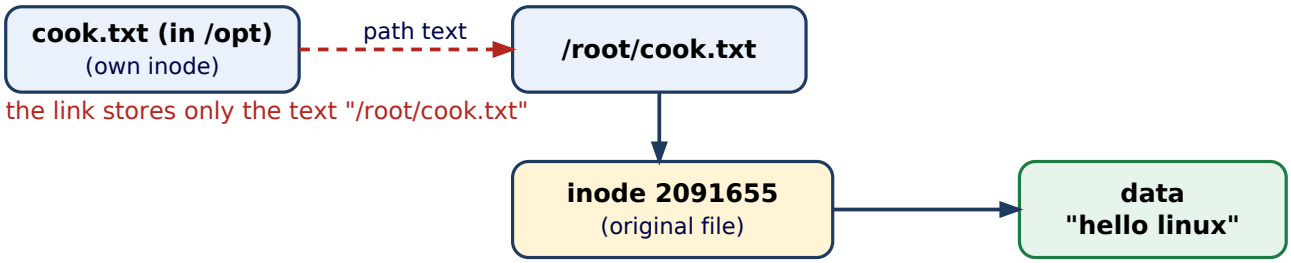
Syntax: **`ln -s TARGET LINK_NAME`** . The **-s** option means symbolic. An existing folder as LINK_NAME again puts the link inside it (ln(1)).

```
ln -s /root/cook.txt /opt
ls -l /opt/cook.txt
```

```
lrwxrwxrwx 1 root root 14 Oct 26 15:54 /opt/cook.txt -> /root/cook.txt
```

Three signs it is a soft link: the line starts with **l** , it shows an arrow **->** , and its inode number is **different** from the target. The size 14 is the length of `"/root/cook.txt"`. **ls -i** shows two different numbers.

Root demonstration; run one command or the other
Creating a link inside /opt needs root privileges. The soft link itself would work even if the target were missing (a dangling link). Remember: **`ln -s /root/cook.txt /opt`** and **`ln /root/cook.txt /opt`** are alternatives: both create `/opt/cook.txt`, so run only one. Practise safely in the demo folder: **`ln -s notes.txt soft.txt`**



A soft link stores a pathname. It can point to a directory, it can cross file systems, and the target need not exist (a **dangling link**). Read it with:

```
readlink /opt/cook.txt
```

```
/root/cook.txt
```

Relative paths
A relative target is read from the folder where the link sits, not from where you typed the command (ln(1)). Our example uses a full path, so the link works from anywhere.
Use **`ln -s -r`** to make relative links automatic.

4. Hard link vs soft link

	Hard link	Soft link
Command	ln /root/cook.txt /opt	ln -s /root/cook.txt /opt
What it is	Another name for the same inode	Small file holding a path
Inode number	Same as the original	Its own, different
Link count	Goes up	Original unchanged
Directories	No	Yes
Other file system	No	Yes
Original removed	Link still works	Link is dangling (broken)
Seen in ls -l	Normal file, count 2 or more	Starts with l, shows ->

Checking inode numbers

```
ls -i notes.txt hard.txt soft.txt   # the simple inode check
```

```
2091655 notes.txt
2091655 hard.txt
2091656 soft.txt
```

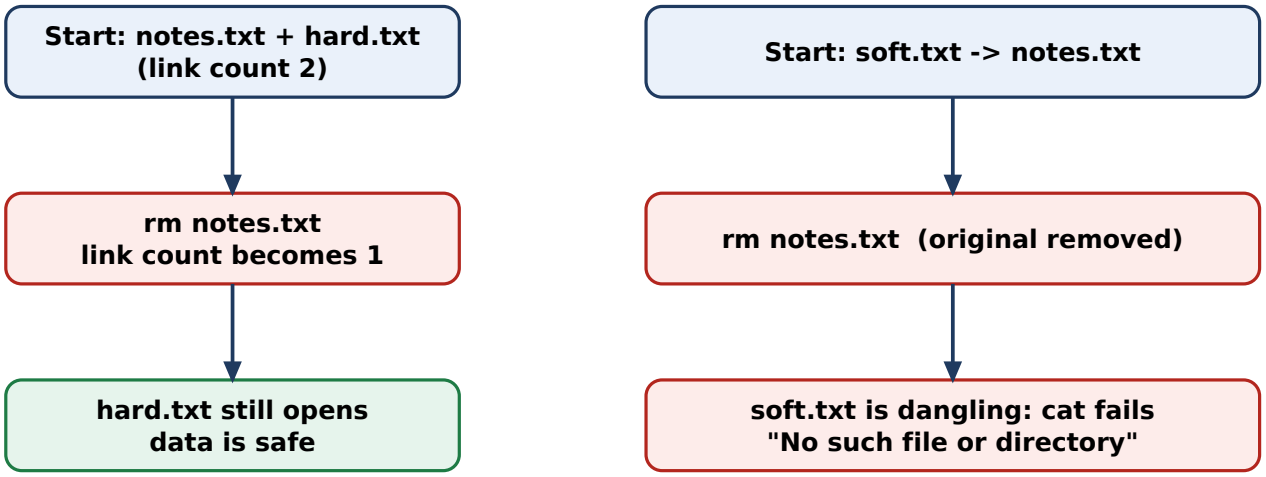
```
stat -c "%i %h %n" notes.txt hard.txt soft.txt
find . -samefile hard.txt           # every name of that inode here
```

```
2091655 2 notes.txt
2091655 2 hard.txt
2091656 1 soft.txt
./hard.txt
./notes.txt
```

ls -i filename prints the inode number of each name: the simple check (ls(1)). **stat format:** `%i` is the inode number, %h the hard link count, %n the file name (stat(1)). **find -samefile name** matches files with the same inode as name (find(1)). It lists hard links, not soft links. Same inode number only means the same file on the **same file system** , because inode numbers are not unique across file systems (symlink(7)).

5. Deleting links and originals

rm removes a **name**. The manual says it unlinks the file. The data is freed only when the **last** name is gone and no program still has the file open (unlink(2)).




```
rm notes.txt          # remove the original name
cat hard.txt           # still works: hello linux
cat soft.txt           # cat: soft.txt: No such file or directory
```

rm -f

`rm -f name` forces removal: it never asks, and ignores files that do not exist (rm(1)). Use it only on a name you are sure about, and only inside the demo folder. The cleanup above uses rm -i instead, so it asks first. To remove a soft link, write the link name with no trailing slash: **`rm -f soft.txt`**. That removes only the link, never the file it points to.

```
rm -i soft.txt hard.txt   # asks before each removal, answer y
cd ~ && rmdir "$demo"      # rmdir only removes an empty folder
```

Deleting a soft link never touches the original. Deleting the original never touches the hard link. A shell trick: use **`rm -i name`** to be asked first.

6. du and df

du (disk USAGE)
adds up the files you point at
answers: how big is this folder?

df (disk FREE)
whole file system
answers: how much space is left?

Same unit options: -h makes sizes human readable (K, M, G).

du: disk usage of files and folders

Command	Meaning
`du`	Usage of each folder under the current one, recursively
`du -h`	Same, with human readable sizes (K, M, G)
`du -s`	Only one total for each argument (-s = summarize)
`du -sh .`	One total for this folder, human readable
`du -sh *`	One total per item in the folder
`du -ah`	All files, not only folders
`du -sh --apparent-size`	Size of the content instead of disk usage
`du -l`	Count hard-linked files each time they appear

```
du -sh "$demo"
du -h --max-depth=1 ~
```

```
4.0K    /tmp/tmp.AbC123
```

--max-depth=1 (or -d 1) shows the folder and its direct sub-folders only. By default du counts a hard-linked file once. Sizes are disk usage, which can differ from file content size (du(1)).

df: free space on file systems

Command	Meaning
`df`	Used and free space of all mounted file systems (1K blocks)
`df -h`	Same, human readable
`df -h .`	Only the file system holding the current folder
`df -i`	Inode usage instead of block usage
`df -T`	Add the file system type

```
df -h .
```

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        28G  7.2G   20G  27% /
```

df reads the whole file system, so it does **not** add up your folders. df cannot show unmounted file systems (df(1)).

	du	df
Looks at	Files and folders	Whole mounted file system
Question	How big is this?	How much is free?
Typical use	Find what is eating space	Check if the disk is full
Common form	du -sh folder	df -h

Full disk: two steps
Run df -h to see which file system is full. Then run du -sh * inside it to find the heavy folder. df -i matters too: a file system can run out of inodes while space remains.

7. find: search live

Form: **`find START-PLACE TESTS ACTION`**. If you give no action, find prints each match.



find START-PLACE TESTS (ACTION). Tests join with AND. Default action prints the path.

Search by name

```
find / -name notes.txt
find ~ -iname "*.txt"      # -iname ignores upper/lower case
```

Common mistake
`-name` matches only the **base name**, not a path. **`find / -name /home/user/notes.txt`** never matches anything, and find warns about it. Use **`-name notes.txt`**. To match a path use **`-path`**. Quote patterns with * so the shell does not expand them.

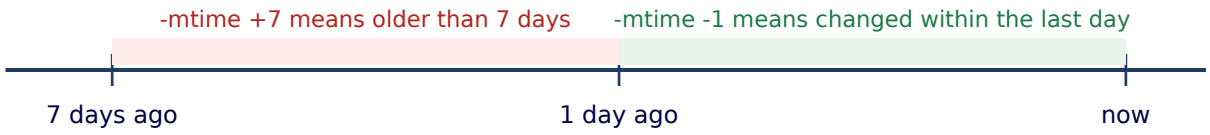
Permission denied lines
Searching / as a normal user prints "Permission denied" for folders you cannot read. The results are still valid. To hide those messages: find / -name notes.txt 2>/dev/null (a shell redirection of error output).

Search by size

```
find / -size +100M            # bigger than 100 MiB
find ~ -type f -size +100M   # only regular files
find ~ -size -10k            # smaller than 10 KiB
```

Part	Meaning
`+100M`	More than 100 mebibytes (+ = greater, - = less, none = exactly)
`Units`	c bytes, k KiB, M MiB, G GiB; no unit means 512-byte blocks
`Rounding`	Sizes round up to the next unit: -size -1M matches only empty files

Search by time (mtime)




```
find ~ -mtime -1       # changed within the last day
find ~ -mtime +7       # changed more than 7 days ago
find ~ -mtime 0        # changed in the last 24 hours
```

mtime is when the file data was last modified. Counting is in 24-hour periods and the fraction is ignored, so **+7** really means at least 8 days (find(1)). Related tests: -atime (last read), -ctime (status change), -mmin (minutes).

Combining tests

```
find ~ -type f -name "*.log" -size +10M -mtime +30
find ~ -type l                  # list soft links
find ~ -links +1 -type f         # files that have extra hard links
```

Tests are joined with AND by default. **-type f** means regular file, **d** directory, **l** symbolic link.

find -delete
`find ... -delete` removes everything that matches, with no question asked. Always run the command first with the default print, read the list, and only then add -delete. Placing -delete first makes it delete everything (find(1)).

find vs locate



find sees changes at once. locate is fast but only knows files from the last updatedb run.

	find	locate
How it works	Walks the real folders when you run it	Searches a saved database of file names
Speed	Slower on big trees	Very fast
Freshness	Always current	Only as new as the last updatedb run
Filters	Name, size, time, type, owner, links and more	File name patterns; mostly name
Can act on results	Yes (-exec, -delete)	No, only lists names
Installed	Part of findutils	May need installing on your system

`locate notes.txt` lists database entries containing that text anywhere in the name. A plain word matches names that contain it; with * ? [] the pattern must match exactly, so start and end with * (locate(1)). **updatedb** refreshes the database (updatedb(1)); a new file may be missing until then.

8. Command chart

Command	What it does
`ln /root/cook.txt /opt`	Hard link (as root): another name for the inode
`ln -s /root/cook.txt /opt`	Soft link (as root): a file that stores a path
`ls -i file`	Show the inode number of each name
`ls -l`	Long list: link count, l and -> on soft links
`stat -c "%i %h %n" f`	Inode, hard link count, name
`find . -samefile f`	All names of the same inode
`readlink link`	Show what a soft link points to
`rm name`	Remove one name (unlink)
`rm -f name`	Remove without asking, ignore missing
`du -sh folder`	Total size of a folder, readable
`du -h --max-depth=1`	Size of each direct sub-folder
`df -h`	Free and used space per file system
`df -i`	Inode usage per file system
`find / -name f`	Search by base name, live
`find / -size +100M`	Files over 100 MiB
`find . -mtime -1`	Changed in the last day
`find ... -delete`	Delete matches (check first!)
`locate f`	Fast search in the database
`updatedb`	Rebuild the locate database

9. Interview questions and answers

Q1. What is an inode?

A file system record that stores a file's details (owner, size, permissions, times) and points to its data. It has a number. File names are labels that point to an inode.

Q2. What is a hard link?

Another name for the same inode, made with ln (no -s). Both names are equal. The file stays until the last name is removed.

Q3. What is a soft link?

A separate small file that stores the path of another file or folder, made with ln -s. It has its own inode.

Q4. Why use hard links?

To give one file a second name without copying its data and without using more space, while keeping the data alive if one name goes.

Q5. Limits of hard links?

They cannot point to directories and cannot cross file systems.

Q6. Limits of soft links?

They can break (dangle) if the target is moved or removed. Otherwise they work across file systems and for directories.

Q7. How do you check if two names are hard links?

Compare inode numbers with ls -i file or stat, or run find . -samefile name. Same inode number on the same file system means the same file (inode numbers are not unique across file systems).

Q8. How do you recognise a soft link?

ls -l starts with l and shows ->. It has a different inode from its target.

Q9. Original file deleted: what happens to each link?

Hard link still works. Soft link becomes dangling and fails with "No such file or directory".

Q10. What does rm -f do to a link?

It removes only that name, without asking. For a soft link the target is untouched.

Q11. Difference between du and df?

du adds up the size of the files and folders you give it. df reports used and free space for the whole mounted file system.

Q12. What does du -sh do?

-s gives one total per argument and -h prints it in K, M or G. du -sh folder = total size of that folder.

Q13. What does df -h show?

Size, used, available, use percent and mount point of each mounted file system, human readable.

Q14. A disk is full. How do you find the cause?

df -h to find the full file system, du -sh * inside it to find the big folders, df -i to check inodes.

Q15. How do you find files bigger than 100 MB?

find / -type f -size +100M. Add 2>/dev/null to hide permission errors.

Q16. What does find -mtime +7 mean?

Files whose data was modified more than 7 full days ago (at least 8 days).

Q17. Why does find / -name /home/a.txt find nothing?

-name matches only the base name. Use -name a.txt, or -path for a full path.

Q18. Difference between find and locate?

find walks the live file system and has many filters. locate reads a prebuilt database, so it is fast but can be out of date until updatedb runs.

Q19. Why can locate miss a new file?

The database has not been updated since the file was created.

Sources

Checked on 26 October 2026. Commands were also run in a temporary test folder to confirm the outputs shown.

https://man7.org/linux/man-pages/man1/ln.1.html

https://man7.org/linux/man-pages/man1/ls.1.html

https://man7.org/linux/man-pages/man7/symlink.7.html

https://man7.org/linux/man-pages/man1/stat.1.html

https://man7.org/linux/man-pages/man1/rm.1.html

https://man7.org/linux/man-pages/man2/unlink.2.html

https://man7.org/linux/man-pages/man1/du.1.html

https://man7.org/linux/man-pages/man1/df.1.html

https://man7.org/linux/man-pages/man1/find.1.html

https://man7.org/linux/man-pages/man1/locate.1.html

https://man7.org/linux/man-pages/man1/updatedb.1.html

https://man7.org/linux/man-pages/man1/mktemp.1.html